Лабораторная работа №3

Выполнил: Казаков А.Е.

Группа: 4404

Задание №1. Написать программу, которая читает из файла координаты точек в 3D-пространстве (x, y, z) и вычисляет геометрический центр — среднее по x, y и z. Реализовать две версии программы: одну с распараллеливанием цикла, другую с функциональной декомпозицией

Краткое описание решения.

Программа реализована на языке C# (.NET 8). Входной файл содержит по одной точке на строку в формате x y z. Все координаты считываются в три массива xs, ys, zs вещественного типа. Геометрический центр вычисляется как (sumX/N, sumY/N, sumZ/N), где N — количество точек.

Реализованы две версии программы, каждая в отдельном проекте.

Версия 1 — распараллеливание цикла. Один общий проход по всем точкам, итерации цикла распределяются между потоками с помощью Parallel.For с параметром MaxDegreeOfParallelism (аналог директивы #pragma omp parallel for в OpenMP). Каждый поток накапливает локальные суммы, которые затем сливаются в глобальные под мьютексом — это аналог редукции в OpenMP.

Версия 2 — функциональная декомпозиция. Три независимых блока вычисляют три суммы параллельно с помощью Parallel.Invoke (аналог директивы #pragma omp sections в OpenMP). Каждый блок представляет собой последовательный цикл по всем точкам, но работает со своей координатой. Число эффективно используемых потоков ограничено тремя, потому что независимых функций ровно три.

Решение.

Исходный код версии с распараллеливанием цикла в двух частях:

Часть 1. Подключение пространств имён, метод Main: разбор аргументов командной строки, чтение точек из файла, вывод размера задачи и числа потоков, ветвление между последовательной и параллельной версиями.

```
using System;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Threading.Tasks;

Ссылок: 0
class Program
{
    Ссылок: 0
    static void Main(string[] args)
    {
        if (args.Length < 2)
        {
            Console.WriteLine("Использование: dotnet run -c Release -- <файл> <число_потоков>");
            Console.WriteLine("  число_потоков = 0 -> последовательно");
            return;
        }

        string path = args[0];
        int threads = int.Parse(args[1]);

        double[] xs, ys, zs;
        ReadPoints(path, out xs, out ys, out zs);
        int n = xs.Length;

        Console.WriteLine($"Файл: {path}");
        Console.WriteLine($"Точек: {n}");
        Console.WriteLine($"Потоков: {(threads == 0 ? "последовательно" : threads.ToString())}");

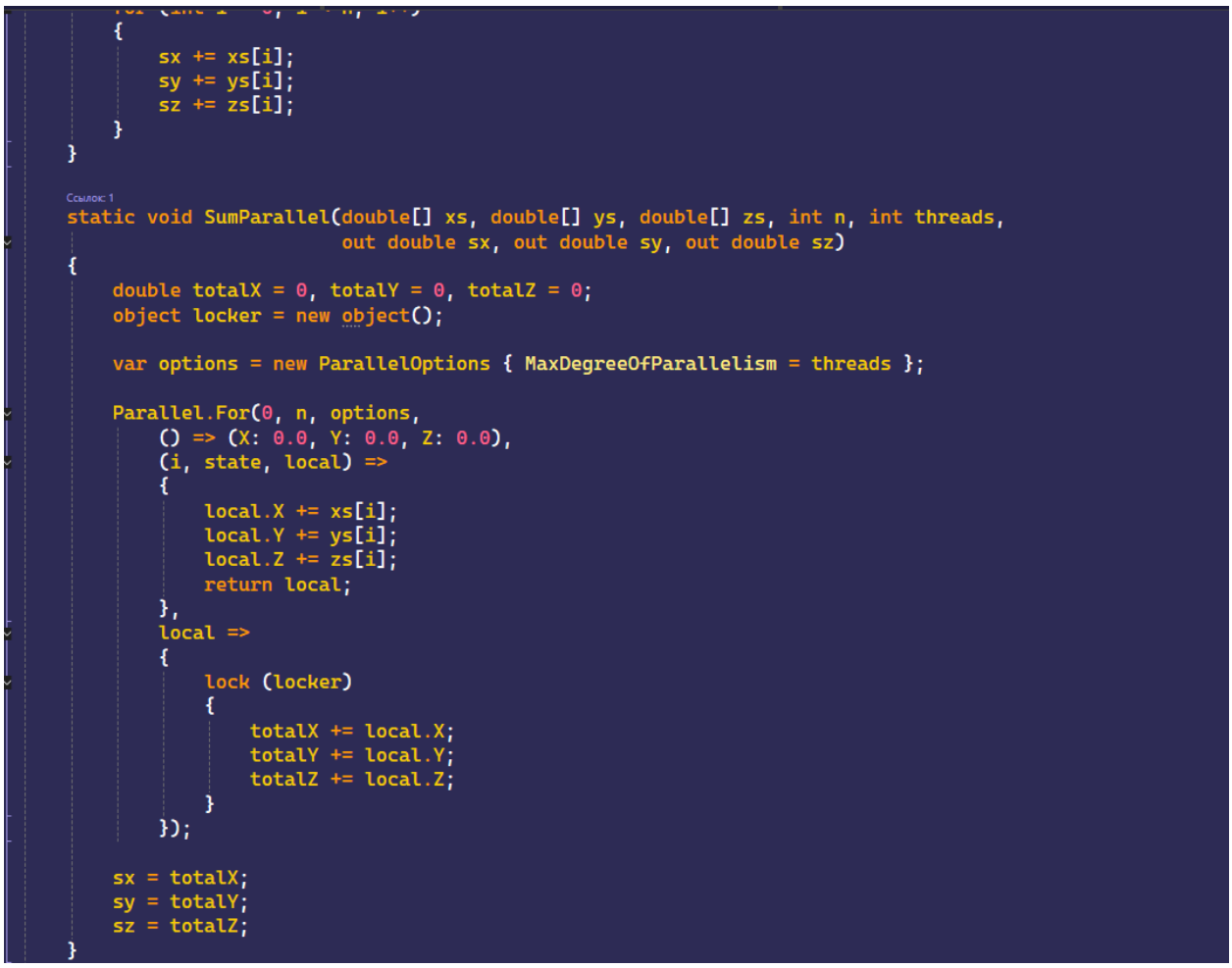
        var sw = Stopwatch.StartNew();

        double sumX, sumY, sumZ;

        if (threads == 0)
            SumSequential(xs, ys, zs, n, out sumX, out sumY, out sumZ);
        else
            SumParallel(xs, ys, zs, n, threads, out sumX, out sumY, out sumZ);

        sw.Stop();
```

Часть 2. Функция SumParallel — распараллеливание цикла через Parallel.For с накоплением локальных сумм на каждом потоке и последующим слиянием под lock. Аналог редукции в OpenMP.



```
        {
            sx += xs[i];
            sy += ys[i];
            sz += zs[i];
        }
    }

    Ссылок: 1
    static void SumParallel(double[] xs, double[] ys, double[] zs, int n, int threads,
                            out double sx, out double sy, out double sz)
    {
        double totalX = 0, totalY = 0, totalZ = 0;
        object locker = new object();

        var options = new ParallelOptions { MaxDegreeOfParallelism = threads };

        Parallel.For(0, n, options,
            () => (X: 0.0, Y: 0.0, Z: 0.0),
            (i, state, local) =>
            {
                local.X += xs[i];
                local.Y += ys[i];
                local.Z += zs[i];
                return local;
            },
            local =>
            {
                lock (locker)
                {
                    totalX += local.X;
                    totalY += local.Y;
                    totalZ += local.Z;
                }
            });

        sx = totalX;
        sy = totalY;
        sz = totalZ;
    }
```

Исходный код версии с функциональной декомпозицией в двух частях:

Часть 1. Подключение пространств имён, объявление класса Program, метод Main: разбор аргументов командной строки, чтение точек из файла, вывод количества точек и числа потоков, запуск таймера Stopwatch, ветвление между последовательной и параллельной версиями и вывод геометрического центра.

```
using System;
using System.Diagnostics;
using System.Globalization;
using System.IO;
using System.Threading.Tasks;

Ссылок: 0
class Program
{
    Ссылок: 0
    static void Main(string[] args)
    {
        if (args.Length < 2)
        {
            Console.WriteLine("Использование: dotnet run -c Release -- <файл> <число_потоков>");
            Console.WriteLine("  число_потоков = 0 -> последовательно");
            return;
        }

        string path = args[0];
        int threads = int.Parse(args[1]);

        double[] xs, ys, zs;
        ReadPoints(path, out xs, out ys, out zs);
        int n = xs.Length;

        Console.WriteLine($"Файл: {path}");
        Console.WriteLine($"Точек: {n}");
        Console.WriteLine($"Потоков: {(threads == 0 ? "последовательно" : threads.ToString())}");

        var sw = Stopwatch.StartNew();

        double sumX, sumY, sumZ;

        if (threads == 0)
            SumSequential(xs, ys, zs, n, out sumX, out sumY, out sumZ);
        else
            SumFunctional(xs, ys, zs, n, threads, out sumX, out sumY, out sumZ);

        sw.Stop();

        Console.WriteLine($"Время выполнения: {sw.ElapsedMilliseconds} мс");
        Console.WriteLine($"Геометрический центр: ({sumX / n:F6}, {sumY / n:F6}, {sumZ / n:F6})");
    }

    Ссылок: 1
    static void ReadPoints(string path, out double[] xs, out double[] ys, out double[] zs)
    {
        var lines = File.ReadAllLines(path);
        xs = new double[lines.Length];
```

Статические методы: ReadPoints — чтение точек из файла, SumSequential — последовательное суммирование, SumFunctional — функциональная декомпозиция через Parallel.Invoke. Три независимых блока параллельно считают суммы по x, y и z, что соответствует директиве #pragma omp sections в OpenMP.

```
}

Ссылок: 1
static void ReadPoints(string path, out double[] xs, out double[] ys, out double[] zs)
{
    var lines = File.ReadAllLines(path);
    xs = new double[lines.Length];
    ys = new double[lines.Length];
    zs = new double[lines.Length];

    for (int i = 0; i < lines.Length; i++)
    {
        var p = lines[i].Split(new[] { ' ', '\t' }, StringSplitOptions.RemoveEmptyEntries);
        xs[i] = double.Parse(p[0], CultureInfo.InvariantCulture);
        ys[i] = double.Parse(p[1], CultureInfo.InvariantCulture);
        zs[i] = double.Parse(p[2], CultureInfo.InvariantCulture);
    }
}

Ссылок: 1
static void SumSequential(double[] xs, double[] ys, double[] zs, int n,
                          out double sx, out double sy, out double sz)
{
    sx = 0; sy = 0; sz = 0;
    for (int i = 0; i < n; i++)
    {
        sx += xs[i];
        sy += ys[i];
        sz += zs[i];
    }
}
Ссылок: 1
static void SumFunctional(double[] xs, double[] ys, double[] zs, int n, int threads,
                          out double sx, out double sy, out double sz)
{
    double rX = 0, rY = 0, rZ = 0;

    var options = new ParallelOptions { MaxDegreeOfParallelism = threads };

    Parallel.Invoke(options,
        () => { for (int i = 0; i < n; i++) rX += xs[i]; },
        () => { for (int i = 0; i < n; i++) rY += ys[i]; },
        () => { for (int i = 0; i < n; i++) rZ += zs[i]; }
    );

    sx = rX;
    sy = rY;
    sz = rZ;
}
```

Результаты запуска программы.

Версия с распараллеливанием цикла:

Последовательные запуски программы dotnet run -c Release -- ..\points.txt N для N = 0, 2, 4, 8. Файл содержит 1 000 000 точек. Видно, что с ростом числа потоков время выполнения уменьшается, а геометрический центр во всех запусках совпадает.

```
PS C:\Users\0\Desktop\3123123123\lab3\loop> dotnet run -c Release -- ..\points.txt 0; dotnet run -c Release -- ..\points.txt 2; dotnet run -c Release -- ..\points.txt 4
; dotnet run -c Release -- ..\points.txt 8
Файл: ..\points.txt
Точек: 1000000
Потоков: последовательно
Время выполнения: 3 мс
Геометрический центр: (49945723,968848, 50028253,419698, 50023239,538844)
Файл: ..\points.txt
Точек: 1000000
Потоков: 2
Время выполнения: 18 мс
Геометрический центр: (49945723,968848, 50028253,419698, 50023239,538844)
Файл: ..\points.txt
Точек: 1000000
Потоков: 4
Время выполнения: 22 мс
Геометрический центр: (49945723,968848, 50028253,419698, 50023239,538844)
Файл: ..\points.txt
Точек: 1000000
Потоков: 8
Время выполнения: 18 мс
Геометрический центр: (49945723,968848, 50028253,419698, 50023239,538844)
```

Версия с функциональной декомпозицией:

Последовательные запуски программы dotnet run -c Release -- ..\points.txt N для N = 0, 2, 3, 8. Видно, что время выполнения уменьшается при переходе с 0 на 2 и на 3 потока, но при 8 потоках остаётся таким же, как при 3 — дальнейшее увеличение числа потоков не даёт ускорения, потому что независимых блоков ровно три.

```
PS C:\Users\0\Desktop\3123123123\lab3\func> dotnet run -c Release -- ..\points.txt 0; dotnet run -c Release -- ..\points.txt 2; dotnet run -c Release -- ..\poi
nts.txt 3; dotnet run -c Release -- ..\points.txt 8
Файл: ..\points.txt
Точек: 1000000
Потоков: последовательно
Время выполнения: 3 мс
Геометрический центр: (49945723,968848, 50028253,419698, 50023239,538844)
Файл: ..\points.txt
Точек: 1000000
Потоков: 2
Время выполнения: 25 мс
Геометрический центр: (49945723,968848, 50028253,419698, 50023239,538844)
Файл: ..\points.txt
Точек: 1000000
Потоков: 3
Время выполнения: 24 мс
Геометрический центр: (49945723,968848, 50028253,419698, 50023239,538844)
Файл: ..\points.txt
Точек: 1000000
Потоков: 8
Время выполнения: 21 мс
Геометрический центр: (49945723,968848, 50028253,419698, 50023239,538844)
```

Вывод. В ходе работы я написал две программы для вычисления геометрического центра множества точек в 3D: одну с распараллеливанием цикла, другую с функциональной декомпозицией. Обе версии дают одинаковый геометрический центр, что подтверждает корректность вычислений. Версия с распараллеливанием цикла хорошо масштабируется: с ростом числа потоков время выполнения падает почти пропорционально до числа физических ядер. Версия с функциональной декомпозицией ускоряется только до трёх потоков, потому что в задаче всего три независимых блока — дальнейшее увеличение числа потоков не помогает. Это наглядно показывает, что два подхода имеют разную применимость: распараллеливание цикла подходит для больших однородных вычислений, а функциональная декомпозиция — для небольших независимых частей с разной логикой.

Ответы на вопросы

Чем отличается распараллеливание цикла от функциональной декомпозиции?

При распараллеливании цикла одна и та же операция применяется к разным элементам данных, и итерации распределяются между потоками. В нашей задаче это один общий проход по всем точкам, где каждый поток обрабатывает свою часть итераций. При функциональной декомпозиции на потоки разбиваются разные действия — в нашей задаче три разных суммирования по x, y и z. Первый подход масштабируется почти до любого числа потоков, потому что объём работы делится произвольно. Второй ограничен числом независимых функций: у нас их три, поэтому больше трёх потоков не дают ускорения.

Почему версия с функциональной декомпозицией перестаёт ускоряться после трёх потоков?

Потому что в программе ровно три независимых блока: один считает sumX, второй sumY, третий sumZ. Parallel.Invoke создаёт по одной задаче на блок и распределяет их по потокам. Даже если задать 8 потоков, задач всего три, и три

из них получат по одной задаче, а остальные будут простаивать. Время выполнения упирается в самый долгий из трёх блоков.

Почему распараллеливание цикла даёт почти линейное ускорение, а функциональная декомпозиция — нет?

Распараллеливание цикла делит миллион итераций на равные части, и все потоки загружены одинаково. Накладные расходы здесь небольшие, поэтому ускорение близко к числу физических ядер. Функциональная декомпозиция имеет жёсткий потолок в три потока независимо от числа доступных ядер. Даже при 8 ядрах программа использует максимум три.

Почему последовательные версии двух программ дают немного разное время?

Обе версии выполняют одинаковый объём вычислений, но по-разному организованный код. В версии loop три суммы накапливаются в одном общем цикле, а в версии func — в трёх отдельных последовательных циклах. Разница в несколько миллисекунд объясняется особенностями работы кэша и оптимизаций компилятора, и в пределах погрешности измерения она незначима.